

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Experiment No.: PBL Q.3 (CO-2)

Student Name: Kottana Roshini

Roll No.: A24126552263

1. TITLE

Express.js Student Server with Routes /, /students and /about

2. AIM

To create a basic Express.js server with the routes /, /students and /about, where /students returns a list of at least five students and /about displays information about the application.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the basics of Node.js and Express.js.
- To create and start an Express server.
- To implement routes using HTTP GET requests.
- To send HTML and JSON responses to the client.
- To handle undefined routes with a 404 response.

4. THEORY

- Node.js is a JavaScript runtime environment that allows JavaScript code to execute outside the browser.
- Express.js is a web framework for Node.js that simplifies creating servers, routes and middleware.
- Routing means deciding how the server responds to a request for a particular path and HTTP method, e.g. `app.get("/students", handler)`.
- HTTP methods: GET requests data from the server, POST sends data to the server, PUT updates and DELETE removes data. This application only reads data, so GET is used.
- `res.send()` sends an HTML/text response and `res.json()` sends data in JSON format, which is commonly used in REST-style servers.
- Middleware is a function that runs during request processing. `app.use()` placed after all routes acts as a catch-all handler for undefined routes (404).
- `app.listen(PORT)` starts the server and listens for requests on the given port.

The basic flow of the application is:

Browser -> HTTP Request -> Express Server -> Matching Route -> Response -> Browser

5. ALGORITHM / PROCEDURE

1. Create a new Node.js project folder.
2. Initialize the project using `npm init -y`.
3. Install Express using `npm install express`.
4. Import the Express module and create an Express application.
5. Create an array containing at least five students.

6. Create a GET route for / that displays a welcome page with links.
7. Create a GET route for /students that returns the student list using res.json().
8. Create a GET route for /about that displays information about the application.
9. Add a catch-all middleware to return a 404 response for undefined routes.
10. Start the server on port 3000 using app.listen().
11. Open <http://localhost:3000/>, /students and /about in the browser.
12. Test valid and invalid routes and verify the responses.

6. PROGRAM

server.js

```
// server.js
// Basic Express.js Student Server
// Routes: / (home), /students (student list), /about (app information)

// Import the Express module (installed using: npm install express)
const express = require("express");

// Create the Express application
const app = express();
// Port number on which the server will listen
const PORT = 3000;

// Pretty-print JSON responses (2 spaces) so they are easy to read
app.set("json spaces", 2);

// Sample student data (at least five students)
const students = [
  { id: 1, name: "Ananya Reddy", department: "CSE", year: 3 },
  { id: 2, name: "Rahul Sharma", department: "CSE", year: 3 },
  { id: 3, name: "Priya Nair", department: "IT", year: 3 },
  { id: 4, name: "Arjun Verma", department: "ECE", year: 3 },
  { id: 5, name: "Kiran Kumar", department: "CSE", year: 3 },
  { id: 6, name: "Sneha Patel", department: "IT", year: 3 }
];

// GET / : home route - shows a welcome page with links to other routes
// (req = incoming request, res = response sent back to the browser)
app.get("/", (req, res) => {
  res.send(`
    <h1>Welcome to the Express.js Student Server</h1>
    <p>Available routes:</p>
    <ul>
      <li><a href="/students">/students</a> - list of students</li>
      <li><a href="/about">/about</a> - about this application</li>
    </ul>
  `);
});

// GET /students : returns the list of students in JSON format
app.get("/students", (req, res) => {
  res.json(students);
});

// GET /about : displays information about the application
app.get("/about", (req, res) => {
```

```

    res.send(`
      <h1>About This Application</h1>
      <p><strong>Application:</strong> Express.js Student Server</p>
      <p><strong>Built with:</strong> Node.js and Express.js</p>
      <p><strong>Purpose:</strong> Demonstrates server creation, routing
      and HTTP GET requests.</p>
      <p><strong>Version:</strong> 1.0</p>
    `);
  });

// Middleware placed after all routes: runs for any URL that did not match
// above, so unknown pages get a 404 (Not Found) response
app.use((req, res) => {
  res.status(404).send("<h1>404 - Page Not Found</h1>");
});

// Start the server and print a message in the terminal
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});

```

7. CODE EXPLANATION

Code	Explanation
require("express")	Imports the Express module.
express()	Creates an Express application.
app.set("json spaces", 2)	Formats JSON responses with indentation for readability.
students array	Stores the sample student data (six students).
app.get("/")	Handles GET requests to the home page.
app.get("/students")	Returns the list of students as JSON using res.json().
app.get("/about")	Displays information about the application.
app.use(...)	Catch-all middleware that sends a 404 response for other routes.
app.listen()	Starts the server on port 3000.

8. TEST CASES AND EXECUTION DATA

The routes were requested one by one and the actual responses were recorded.

Test Case	Request	Expected Result	Actual Result	Status
TC-01	GET /	Status 200; home page with welcome heading	Status 200; heading: "Welcome to the Express.js Student Server"	Pass
TC-02	GET /students	Status 200; JSON list of at least 5 students	Status 200; application/json; 6 students (Ananya Reddy ... Sneha Patel)	Pass
TC-03	GET /about	Status 200; information about the application	Status 200; heading: "About This Application"	Pass
TC-04	GET /contact (undefined route)	Status 404; page not found message	Status 404; "404 - Page Not Found"	Pass

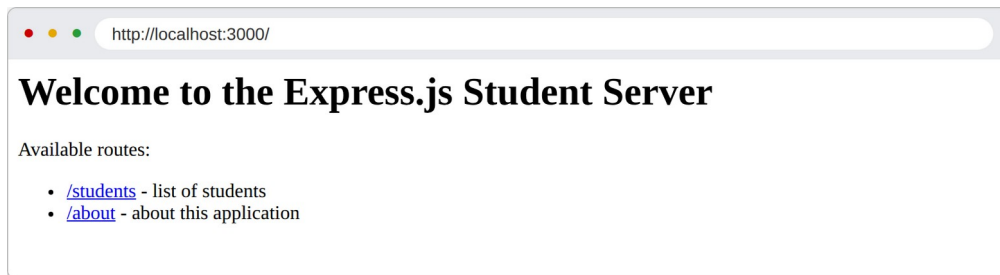
Test Case	Request	Expected Result	Actual Result	Status
TC-05	POST /students (method not defined)	Status 404; route accepts GET only	Status 404; "404 - Page Not Found"	Pass

9. OUTPUT

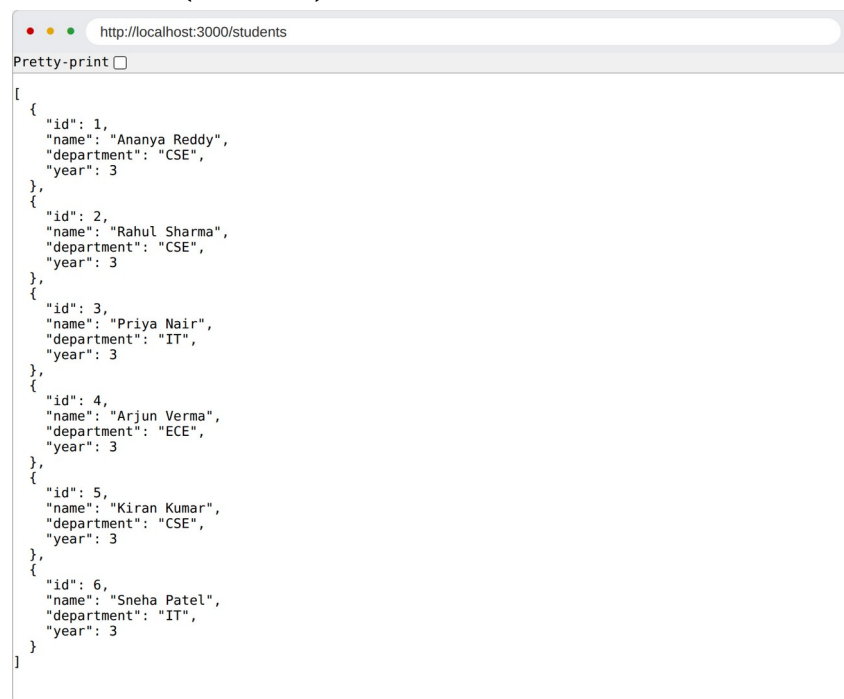
Output 1: Server execution

```
Terminal
$ node server.js
Server running at http://localhost:3000
```

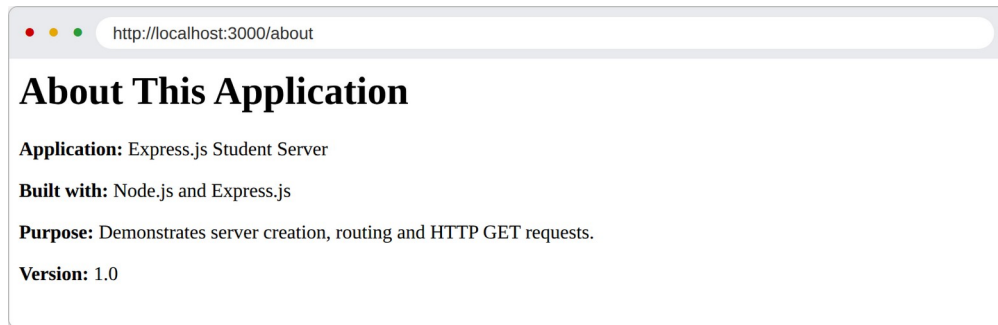
Output 2: Home route (/)



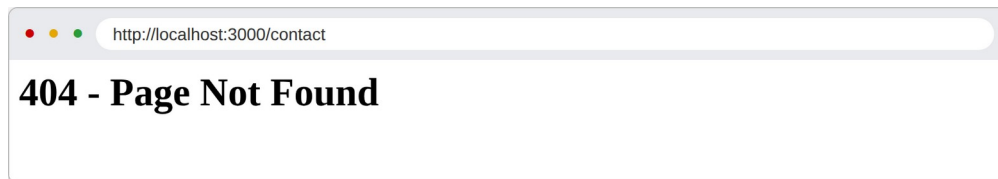
Output 3: Students route (/students)



Output 4: About route (/about)



Output 5: Undefined route (/contact) returns 404



10. RESULT

Thus, a basic Express.js server was successfully created with the routes /, /students and /about. The /students route returns a list of six students, the /about route displays information about the application, and the server was verified using multiple test cases.